Level 0 (1 Hour - 4 Questions)

Focus: Conditional Logic, Basic Math, and Real-time application.

Q1: Smart Agriculture - Precision Irrigation

Title: Precision Irrigation Assistant **Problem:** In a smart farm, sensors track soil moisture to optimize water usage. Your task is to calculate the final water volume required for a field based on its area and moisture level.

- **Base Requirement:** 5 Liters of water per square meter of `Area`.
- **Conditions:**
  - If `Moisture < 30.0%`: Increase total volume by **25%** due to high dryness.
  - If `30.0% <= Moisture <= 60.0%`: Use the base volume.
  - If `Moisture > 60.0%`: Decrease total volume by **50%** (already humid).
  - If `Moisture > 80.0%`: Total volume required is **0 Liters**. **Input Format:**
- Area (float)
- Moisture Percentage (float) **Output Format:**
- `Total: [Volume]L` (rounded to 1 decimal place)

Example:

- Input: `100.0`, `25.0`
- Calculation: Base = 500. Add 25% (125) = 625.0.
- Output: `Total: 625.0L`

Q2: EV Revolution - Dynamic Range Estimator

Title: EV Intelligent Range Predictor **Problem:** An Electric Vehicle (EV) has a maximum range of 400km on a full charge (100%). However, performance depends on cargo and cabin climate.

- **Initial Range:** `(Battery % / 100) * 400` km.
- **Cargo Impact:** For every **1.0 kg** of cargo, the range decreases by **0.05 km**.
- **AC Impact:** If the Air Conditioning (AC) is ON, the *remaining* range (after cargo adjustment) is reduced by **15%**. **Input Format:**
- Battery Percentage (int)
- Cargo Weight in kg (float)
- AC Status (1 for ON, 0 for OFF) **Output Format:**
- `Predicted Range: [Value] km` (rounded to 2 decimal places)

Example:

- Input: `80`, `100.0`, `1`
- Calculation: Initial = 320km. Cargo = 320 - (100 * 0.05) = 315km. AC = 315 * 0.85 = 267.75.
- Output: `Predicted Range: 267.75 km`

Q3: Logistics - Gig Economy Earnings

Title: Last-Mile Delivery Pay Calculator **Problem:** A delivery partner's earnings for a single trip are calculated based on distance and customer satisfaction.

- **Base Pay:** Rs. 40 flat.
- **Distance Pay:**
  - First 5 km: Rs. 10 per km.
  - Beyond 5 km: Rs. 15 per km (for the additional distance).
- **Pro Bonus:** If the customer rating is **4.8 or higher**, the *entire* amount (Base + Distance) receives a **20% bonus**. **Input Format:**
- Distance in km (float)
- Customer Rating (float) **Output Format:**
- `Earnings: Rs. [Value]` (rounded to 2 decimal places)

Example:

- Input: `8.0`, `4.9`
- Calculation: Base = 40. Distance = (5 * 10) + (3 * 15) = 95. Total = 135. Bonus = 135 * 1.2 = 162.00.
- Output: `Earnings: Rs. 162.00`

Q4: Urban Mobility - Smart Toll/Parking

Title: Smart City Parking Fee **Problem:** A smart parking lot uses automated billing based on vehicle type and duration.

- **First 2 Hours:** Rs. 30 (total).
- **Every Hour After:** Rs. 20 per hour.
- **Vehicle Type:**
  - If it's an **Electric Vehicle (EV)**, apply a flat **Rs. 15 discount** on the final total.
  - If the duration is more than **12 hours**, apply a **10% surcharge** on the final amount (before EV discount). **Input Format:**
- Total Hours (int)
- Is EV (1 for Yes, 0 for No) **Output Format:**
- `Final Fee: Rs. [Value]` (rounded to 2 decimal places)

Example:

- Input: `5`, `1`
- Calculation: 30 + (3 * 20) = 90. No surcharge (5 < 12). Discount = 90 - 15 = 75.00.
- Output: `Final Fee: Rs. 75.00`

Level 1 (2 Hours - 2 Questions)

Focus: Data Structures, Grid Logic, and Security Algorithms.

Q1: Smart Warehouse - Autonomous Robot Navigator

Title: Robot Path Validator **Problem:** A robot in a 10x10 grid (starting at 0,0) moves based on a command string ('U', 'D', 'L', 'R').

1. You are given a target location `(TX, TY)`.
2. You are given `N` obstacles at specific coordinates.
3. If the robot:
   - Hits a boundary (0-9 range), it stops and reports: `OUT_OF_BOUNDS at (x,y)`
   - Hits an obstacle, it stops and reports: `OBSTACLE at (x,y)`
   - Reaches the target *at any point* during the moves: `TARGET_REACHED`
   - Finishes all moves without reaching target: `INCOMPLETE at (x,y)` **Input Format:**

- Target X, Target Y (two ints)
- Number of Obstacles (int)
- Obstacle coordinates (list of x y pairs)
- Moves String (string) **Output Format:**
- The status message as defined above.

Example:

- Input: `2 2`, `1`, `1 1`, `RRUU`
- Output: `TARGET_REACHED` (Path: (0,0) -> (1,0) -> (2,0) -> (2,1) -> (2,2). No obstacles hit).

Q2: Healthcare - Patient Vital Monitoring System

Title: Real-time Health Alert System **Problem:** A wearable health device monitors a patient's Heart Rate (HR) every minute. To prevent false alarms, the system must detect sustained abnormalities over a "3-minute window."

- **EMERGENCY:** Triggered if HR is **above 150** for 3 consecutive minutes.
- **WARNING:** Triggered if HR is **above 120** for 3 consecutive minutes (and not an Emergency).
- **STABLE:** If neither of the above conditions are met.
- If multiple conditions are met, the **highest** priority alert (Emergency > Warning) must be reported. **Input Format:**
- 10 integers representing HR readings for each minute. **Output Format:**
- `Alert: EMERGENCY`, `Alert: WARNING`, or `Status: STABLE`

Example:

- Input: `80 90 125 130 128 100 90 155 160 165`
- Calculation:
  - Minutes 3-5: All > 120 (Warning triggered)
  - Minutes 8-10: All > 150 (Emergency triggered)
- Output: `Alert: EMERGENCY`